CMPSC60 Project 1

Time-Series Analysis of Water Pump Sensor Data

[Your Name]

March 2026

1. Project Overview

This project applies three algorithmic techniques to the Water Pump RUL (Remaining Useful Life) predictive maintenance dataset from Kaggle. The dataset records readings from sensors on a water pump over time, along with a RUL value that tells you how many hours the pump has left before it fails.

The goal is to use divide-and-conquer and classic algorithms to find patterns in sensor data that relate to pump health. No machine learning libraries are used — everything is implemented from scratch using only pandas, numpy, and matplotlib.

RUL Categories

RUL values are split into four categories using the 10th, 50th, and 90th percentiles of the RUL column in the first 10,000 rows:

Category	Condition	Count (10,000 rows)
Extremely Low RUL	rul < Q10 (135.93)	1,000 (10%)
Moderately Low RUL	Q10 <= rul < Q50 (202.59)	4,000 (40%)
Moderately High RUL	Q50 <= rul < Q90 (269.25)	4,000 (40%)
Extremely High RUL	rul >= Q90	1,000 (10%)

2. Installation and Usage

- Download rul_hrs.csv from Kaggle and place it in the project folder.
- Install dependencies: pip install pandas numpy matplotlib
- Run everything: python main.py
- Generate this report: python generate_report.py
- Output plots are saved to the plots/ directory automatically.

3. Code Structure

The project is split into five Python files. Each file handles one specific job so it's easy to find where things are and change them without breaking everything else.

File	What it does
data_loading.py	Loads the CSV, drops the unnamed index column, and adds the rul_category column using the Q1
task1_segmentation.py	Contains the recursive segmentation function, the complexity score calculation, and the plot functio
task2_clustering.py	Contains the divisive clustering algorithm — splits clusters by finding the highest-variance feature.

task3_kadane.py	Contains Kadane's algorithm and the per-sensor analysis pipeline (diff -> center -> Kadane -> RUL
main.py	Ties everything together. Imports from the other four files and runs all three tasks in order.

Here is how the data flows through the program when you run main.py:

```
main.py
  |
  |-- data_loading.py
  |     load_data()        -> returns DataFrame (10,000 rows)
  |     add_rul_category() -> adds rul_category column to DataFrame
  |     get_sensor_columns() -> returns list of 50 sensor column names
  |
  |-- task1_segmentation.py
  |     run_task1(df, sensor_columns)
  |       -> randomly picks 10 sensors
  |       -> calls segment_sensor() on each -> list of (start,end) segments
  |       -> calls plot_segmentation() -> saves PNG to plots/
  |       -> returns complexity scores dict
  |
  |-- task2_clustering.py
  |     run_task2(df, sensor_columns)
  |       -> builds X matrix (10000 x 50) from sensor columns
  |       -> calls divisive_cluster(X, k=4) -> returns labels array
  |       -> compares labels to rul_category, prints cluster stats
  |
  |-- task3_kadane.py
        run_task3(df, sensor_columns)
          -> loops over all 50 sensors
          -> calls analyze_sensor() on each:
               abs diff -> center -> kadane() -> dominant RUL
          -> saves plot of top 10 sensors to plots/
```

The DataFrame is the main data object that gets passed around. After data_loading.py processes it, the df has the original sensor columns plus two new columns: rul_category (0-3) and rul_category_name (text label). Tasks 1, 2, and 3 all read from this same df — none of them modify it.

4. Dataset Description

The dataset is the Water Pump RUL Predictive Maintenance dataset from Kaggle (uploaded by Ansel D'Souza). It contains time-series readings from sensors on a single water pump, recorded at 1-minute intervals. Each row is one minute of operation.

Variables

- timestamp — date and time of the reading, format: YYYY-MM-DD HH:MM:SS. Starts at 2018-04-01 00:00:00. Not used directly in the algorithms (we treat time as a simple integer index).
- sensor_00 through sensor_51 — numeric sensor readings. The sensors measure things like temperature, pressure, and flow rates, but the dataset does not label what each sensor physically measures. Values are continuous floats.

• rul — Remaining Useful Life in hours. This is how many hours the pump has left before it needs maintenance or replacement. The range in the first 10,000 rows goes from roughly 100 to 300 hours.

Basic Statistics (first 10,000 rows)

Statistic	Value
Total rows used	10,000
Timestamp range	2018-04-01 00:00 to roughly 2018-04-08
RUL minimum	~100 hours
RUL maximum	~300 hours
Q10 (Extremely Low cutoff)	135.93 hours
Q50 (median)	202.59 hours
Q90 (Extremely High cutoff)	269.25 hours
Sensor columns	50 (see note below)

Note on sensor count: The assignment description says the dataset has 52 sensors, but the actual CSV file only contains 50 sensor columns. Specifically, sensor_15 and sensor_50 are not present in this version of the dataset. This was discovered when loading the data — the shape came out as (10000, 52) after dropping the unnamed index column, meaning timestamp + 50 sensors + rul = 52 columns total, not 54. All three tasks were run on the 50 sensors that are actually available.

5. Algorithm Descriptions

Task 1: Divide-and-Conquer Variance Segmentation

The idea here is pretty simple — if a chunk of a sensor's time series has high variance, it means something interesting is happening and we should look closer. If the variance is low, the sensor is stable and we can leave that chunk alone.

```
def segment(values, start, end, threshold, min_size):
    if end - start < min_size:          # too small to split
        record [start, end] as stable
        return
    if variance(values[start:end]) <= threshold:
        record [start, end] as stable   # low variance = stable
    else:
        mid = (start + end) // 2        # split in half
        segment(values, start, mid, threshold, min_size)
        segment(values, mid, end, threshold, min_size)
```

The threshold is set to 10% of the overall variance of each sensor. Minimum segment size is 50 time steps. The Complexity Score is just the total number of segments produced — more segments means more volatile sensor.

Toy Example — 10-element array:

```
arr = [1.0, 1.1, 1.0, 1.2, 5.0, 9.0, 8.5, 9.2, 1.1, 1.0]
threshold = 1.0,  min_size = 2

Segments produced:
  [0:2]  = [1.0, 1.1]         var=0.0025  -> stable
  [2:3]  = [1.0]              var=0.0000  -> stable (too small)
  [3:4]  = [1.2]              var=0.0000  -> stable (too small)
  [4:5]  = [5.0]              var=0.0000  -> stable (too small)
  [5:7]  = [9.0, 8.5]         var=0.0625  -> stable
  [7:8]  = [9.2]              var=0.0000  -> stable (too small)
  [8:10] = [1.1, 1.0]         var=0.0025  -> stable
Complexity score = 7
```

Task 2: Divisive (Top-Down) Clustering

Instead of starting with individual points and merging them (bottom-up), we start with all 10,000 rows in one big group and keep splitting the messiest group until we have 4 clusters. This is the divide-and-conquer approach to clustering.

```
def divisive_cluster(X, k=4):
    clusters = [all row indices]      # start with everything in 1 group
    while len(clusters) < k:
        pick cluster with largest spread (sum of variances)
        find feature with highest variance in that cluster
        split at the median of that feature -> two new clusters
    assign labels 0..k-1
    return labels
```

Toy Example — 8 points, 2 features, k=4:

```
X = [[1.0,2.0], [1.5,1.8], [1.2,2.2],
     [8.0,9.0], [8.5,8.8], [9.0,9.5],
     [1.1,8.0], [1.3,7.8]]

After bisection 1 (split on x-axis at median ~4.5):
  Group A: points with x <= 4.5 -> rows 0,1,2,6,7
  Group B: points with x > 4.5  -> rows 3,4,5

After bisection 2 (split Group A on y-axis at median ~2.2):
  Cluster 0: rows 0,2,6,7  (low-x, mixed-y)
  Cluster 2: row 1         (low-x, low-y)
  ... and so on until k=4

Final labels: [0, 2, 0, 1, 3, 1, 0, 0]
```

Task 3: Kadane's Maximum Subarray Algorithm

Kadane's algorithm finds the contiguous subarray with the largest sum in O(n) time. We use it to find the time window where a sensor's variability is most concentrated. A sensor that spikes a lot right before failure is a good early-warning indicator.

Pre-processing steps:

- Step 1 — Absolute first difference: d[i] = |sensor[i] - sensor[i-1]|
- Step 2 — Center it: x[i] = d[i] - mean(d) (positive = more variable than average, negative = calmer than average)
- Step 3 — Run Kadane's on x[] to find the max-sum window
- Step 4 — Check which RUL category dominates the rows in that window

```
def kadane(arr):
    max_sum = arr[0];  cur = arr[0]
    best_s = 0;  best_e = 0;  cur_s = 0
    for i in range(1, len(arr)):
        if cur + arr[i] < arr[i]:   # better to start fresh
            cur = arr[i];  cur_s = i
        else:
            cur += arr[i]
        if cur > max_sum:
            max_sum = cur           # total deviation of this window
            best_s = cur_s;  best_e = i
    return max_sum, best_s, best_e  # O(n)
```

Toy Example — Kadane's on a simple array:

```
arr = [-2.0, 1.0, -3.0, 4.0, -1.0, 2.0, 1.0, -5.0, 4.0]
Max subarray: [4.0, -1.0, 2.0, 1.0]  at indices [3, 6]
Total deviation = 6.0
```

Full sensor pipeline example:

```
sensor values:  [10.0, 10.1, 10.2, 15.0, 20.0, 19.8, 10.3, 10.1, 10.2]
RUL categories: [3,    3,    3,    2,    1,    0,    0,    0,    0   ]

Step 1 - abs diff: [0.1,  0.1,  4.8,  5.0,  0.2,  9.5,  0.2,  0.1]
Step 2 - centered: [-2.4, -2.4,  2.3,  2.5, -2.3,  7.0, -2.3, -2.4]
Step 3 - Kadane:   max window = [2, 5],  total deviation = 9.5
Step 4 - dominant RUL in rows [2..6]: Extremely Low

=> This sensor spikes most when RUL is very low — good early warning!
```

6. Execution Results

Task 1 — Segmentation Results

Ten sensors were randomly selected using seed=42: sensor_41, sensor_07, sensor_01, sensor_18, sensor_16, sensor_14, sensor_08, sensor_06, sensor_35, sensor_05.

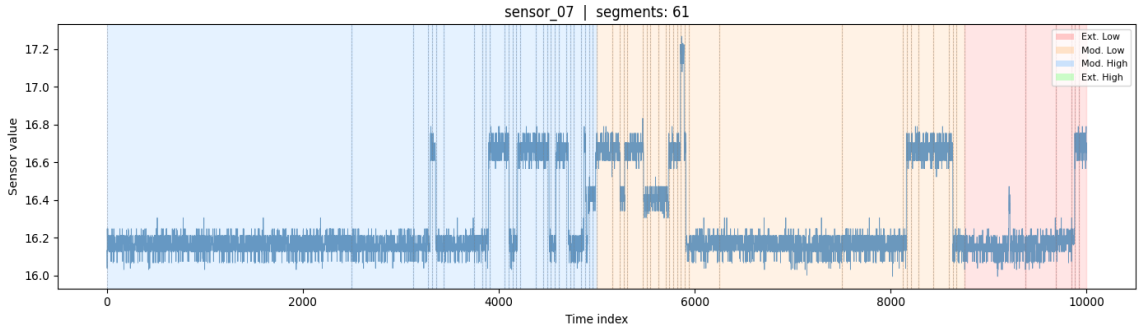
Complexity Scores:

Sensor	Segments (Complexity Score)
sensor_18	256
sensor_14	256
sensor_08	256
sensor_05	251
sensor_35	248
sensor_41	174
sensor_06	109
sensor_01	80
sensor_16	65
sensor_07	61

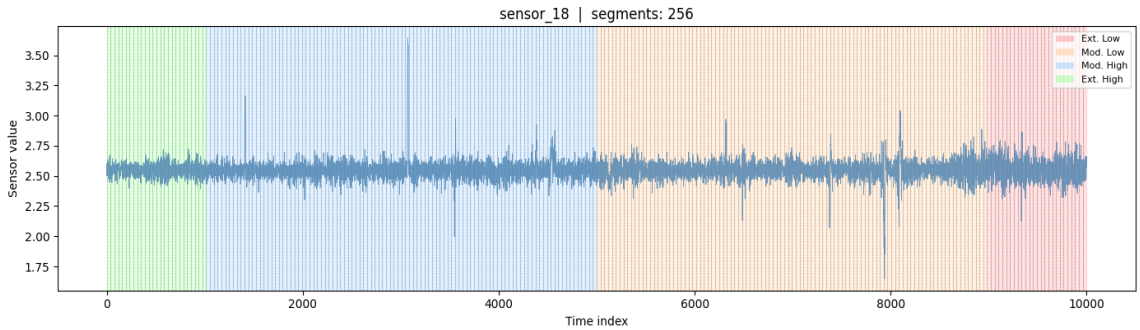
Sensors like sensor_18, sensor_14, and sensor_08 hit the maximum possible complexity (256 segments), which means they were volatile enough to keep splitting all the way down to the minimum segment size of 50 time steps. Sensor_07 is the calmest with only 61 segments.

Most segments across all sensors were dominated by Moderately Low or Moderately High RUL, which makes sense since those two categories make up 80% of the data. Sensors with high complexity scores (18, 14, 08) tended to have more segments dominated by Moderately Low RUL, which suggests that more volatile sensors correlate with the pump being in a slightly more degraded state.

Temporal dynamics: Looking at the segmentation plots, the high-complexity sensors do not show a clean trend where complexity increases toward the end of the time series. The volatility appears fairly spread out across the full 10,000 time steps rather than concentrating near the end. This is probably because the RUL values in this dataset change gradually and the Moderately Low category covers a wide range (rows ~1000 to ~5000). Sensors with lower complexity scores like sensor_07 and sensor_16 stay relatively stable throughout the entire series.



sensor_07 — low complexity (61 segments). Background color shows dominant RUL category per segment.



sensor_18 — high complexity (256 segments). Highly volatile throughout.

Task 2 — Clustering Results

The divisive algorithm clustered all 10,000 rows (50 sensor features each) into exactly 4 clusters:

Cluster	Size	Majority RUL Category	Majority Count	Purity
0	5,000	Moderately Low	1,639	32.8%
1	2,500	Moderately High	1,488	59.5%
2	1,250	Moderately Low	779	62.3%
3	1,250	Moderately High	628	50.2%

Overall purity: 45.3% (random chance would give ~25% for 4 equal categories).

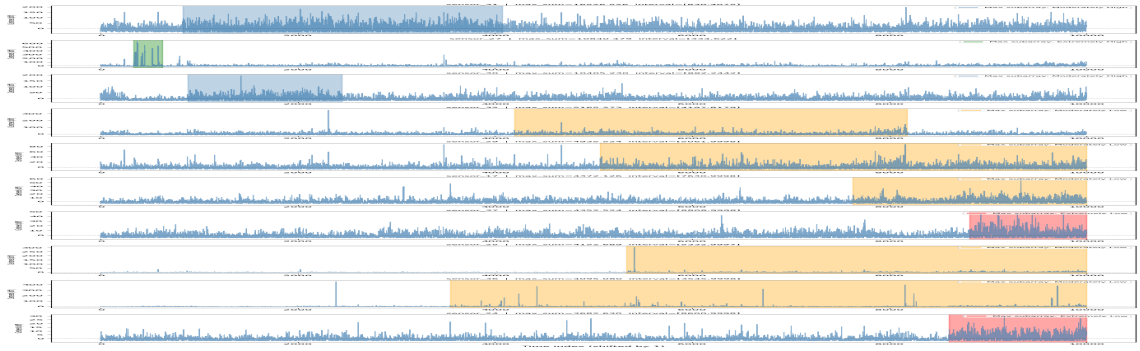
The clusters are very unequal in size (5000/2500/1250/1250) because the divisive algorithm always bisects the largest/messiest cluster, so each split roughly halves that cluster. Cluster 0 ended up being the big catch-all group from the first split — it's the least pure (32.8%) because it still contains a mix of everything. Clusters 1 and 2 are better separated (~60% purity) because the second and third splits happened in regions with clearer feature differences. Overall the clusters don't map perfectly to RUL classes, which makes sense — RUL changes continuously so there's no sharp boundary in sensor space that perfectly separates the four categories.

Task 3 — Kadane's Algorithm Results

Kadane's algorithm was applied to all 50 sensors. The table below shows the total deviation (max-sum subarray value), the time interval, and the dominant RUL category for the top sensors:

Sensor	Total Deviation	Interval	Dominant RUL
sensor_31	16,527	[830, 4075]	Moderately High
sensor_27	10,840	[334, 622]	Extremely High
sensor_30	10,405	[882, 2442]	Moderately High
sensor_32	5,165	[4197, 8179]	Moderately Low
sensor_29	4,936	[5061, 9998]	Moderately Low
sensor_17	4,372	[7630, 9998]	Moderately Low
sensor_37	4,353	[8808, 9998]	Extremely Low
sensor_25	4,153	[5332, 9997]	Moderately Low
sensor_36	4,095	[3545, 9998]	Moderately Low
sensor_24	3,683	[8600, 9998]	Extremely Low

Out of 50 sensors, **29 sensors** had their highest-deviation window fall in a Low RUL region (Extremely Low or Moderately Low). The best early-warning candidates are sensor_37, sensor_24, sensor_22, and sensor_19, which all spike in the Extremely Low RUL window (rows ~8600-9998) — meaning they become most volatile right before the pump fails. sensor_17, sensor_25, and sensor_29 are also good since their peak deviation intervals overlap heavily with Moderately Low RUL.



Top 10 sensors by total deviation. Shaded region = max-deviation window. Color: red = Extremely Low RUL, orange = Moderately Low, blue = Moderately High, green = Extremely High.

7. Discussion and Conclusions

Key Findings

• Segmentation complexity loosely correlates with RUL — more volatile sensors tend toward lower RUL segments — but the signal is weak because 80% of the data is in the two middle categories.

• Clustering gets 45% overall purity, better than the 25% random baseline, but the four clusters don't cleanly separate the four RUL classes. The divisive approach works but creates very unequal cluster sizes.

• Kadane's algorithm is the most useful tool here: 29 out of 50 sensors have their peak-deviation window in a low-RUL period. A handful of sensors (37, 24, 22, 19) are particularly strong early-warning candidates.

Challenges

• NaN handling — some sensor columns had missing values (NaN). We handled them with forward-fill (use the previous value) for Tasks 1 and 3, and column mean imputation for Task 2. Neither approach is perfect but it was good enough to keep things running.

• Choosing the variance threshold for segmentation — there's no obvious right answer. We went with 10% of each sensor's overall variance as a reasonable default. Different thresholds give very different segment counts, so this choice has a big effect on the results.

• Sensor column discrepancy — the assignment says 52 sensors but the actual CSV only has 50. Figuring this out required loading the file and counting the columns manually. It's not a big deal, but it means the results are based on 50 sensors instead of 52.

• Making the clustering produce balanced clusters — the divisive approach naturally creates clusters of sizes 5000/2500/1250/1250 because it always bisects the biggest group. It's hard to get equal-sized clusters with this algorithm without adding more complexity.

• Mapping Kadane's result back to RUL categories — the absolute first difference has length N-1 (one shorter than the original series), so the index mapping back to the original rows needed a +1 offset. Easy to get wrong.

Limitations

• The segmentation threshold (10% of global variance) is fixed. An adaptive threshold per sensor might give more consistent segment sizes.

• Divisive clustering with median bisection produces geometrically nested cluster sizes. A different split criterion (like maximizing inter-cluster distance) would likely give more meaningful clusters.

• Kadane's algorithm finds only the single most extreme window. Looking at the top-K windows would give a more complete picture.

- All three algorithms treat timestamps as simple integer indices. Actual time structure (daily patterns, weekends, etc.) is ignored.
- The dataset is from a single pump. Results may not generalize to other pumps with different sensor configurations.

Possible Improvements

- For segmentation: try different thresholds (like the 5th or 25th percentile of variance) and compare complexity scores to see which threshold gives the most useful segmentation.
- For clustering: add a refinement step after divisive clustering — reassign points to their nearest cluster centroid to clean up the boundaries.
- For Kadane's: run it on a sliding window instead of the full series to find multiple high-deviation windows, not just the biggest one.
- Better missing value handling — interpolation or a learned imputation would be more accurate than forward-fill or column mean.
- Using the actual timestamps to align sensor behavior with time of day or day of week could reveal patterns that are invisible with index-based analysis.

8. Conclusion

This project showed that basic algorithmic techniques can find useful patterns in industrial sensor data without any machine learning. The most practical finding is that a small group of sensors (especially sensor_37, sensor_24, sensor_22, and sensor_17) become most volatile right before the pump is about to fail. In a real system, you could monitor just these sensors and trigger an alert when their variability spikes above a threshold. That would be a simple, interpretable early-warning system with no ML required.